# Introduction to OOPs.

→ How OOPs Helps solving real world problems ?

OOPs Models code based on real-world thing (like a car, employee, Students.) Each real-world object becomes a class or object in programming. This makes it easy to think, plan, and develop software just like real life.

→ class :- A class is a blue print / template. It define what an object will have (its data) and what it can do (its actions).

A car design Made by a company before building real cars. It define. " Every car will have color, speed, fuel type, and can drive, brake etc.

→ object :- An object is a real-world example created from a class

→ Attributes (Also called properties or data members) These are variable inside the class that holds data.
Attributes define the characteristic of an object.
Model No, color, speed, company etc.

→ Methods (Also called functions inside class).
Methods define behaviour / action of an object. These are the things the car can do.
Ex :- Start(), drive(), brake() etc.

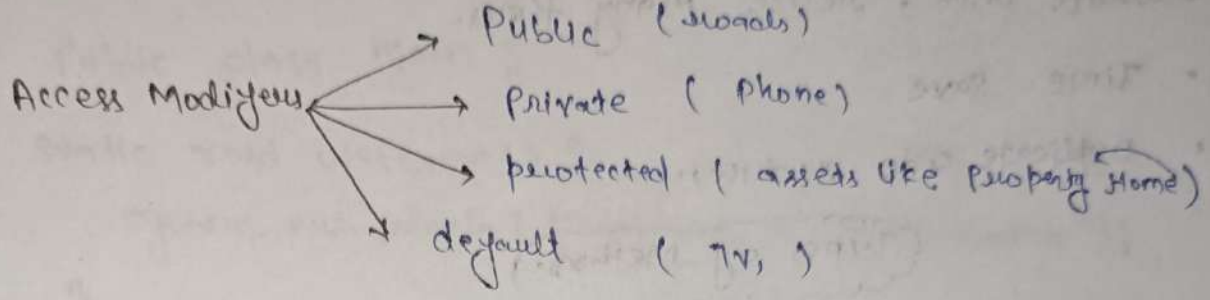
3 characteristics of an objects.

| | | |
|---|---|---|
| (1) Identity | unique name of objects | BMW, Toyata |
| (2) State | attributes | red, model no |
| (3) behaviour | what the can do | accelerate, |
| | | |

## Create a class.

- Access Modifiers → 4
- class Keyword
- Class Name
- Body name.

```
Public class car {
    // class Body
}
```




```
Public class Person {
    int age = 20;
Public static Void Main (String [] args) {
    Person Rohan = new person ();
    System.out.println (Rohan.age); } };
```

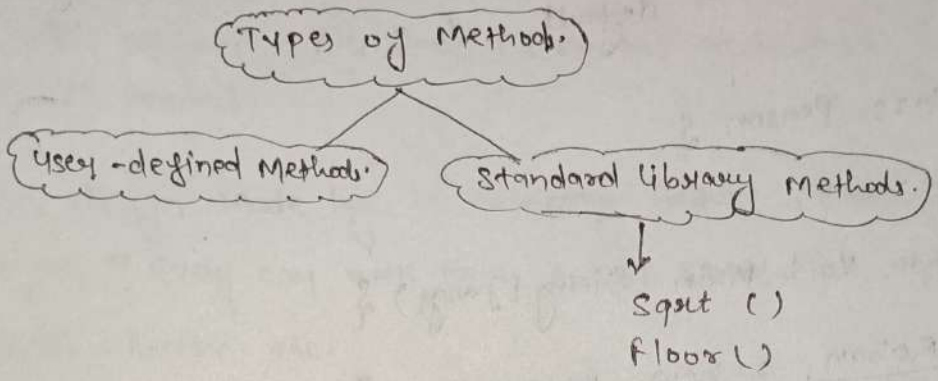
Person → class name
Rohan → object name
new → keyword

Lecture:-12

Java Method:- It is block of code performing some action when runs only when it is called.

→ Why are methods important in Java.?

- Write once, reuse many times.
- Time Save
- Duplicate code reduces.




How to declare Methods:-

```
Pubelic class Main {
    Public int sum (int a int b) {
      // code to be executed
    }
}.
```

Access Modifier → Return type → Method Name → (Parameters).

(1) Method Signature :-

Sum (int a int b)

↓

Method name + Parameter list.

(2) Access specifier.

(i) Public (ii) Private (iii) Protected (iv) default

call a method :- sum();

```
Public class Main {
Static void welcome() {
    system.out.println("Welcome to physics wallah");
}
Public static void main(String[] args) {
    welcome();
} }.
```

Questions:- Java program to add two numbers using methods.

```
class Algebra {
    int add(int a, int b) {
     int ans = a+b;
     return ans;
    }
}
Public class Main {
    Public static void Main(String[] args) {
     Algebra obj = new Algebra();
     Scanner sc = new Scanner(system.in);
     int a = sc.nextInt();
     int b = ,,  ,,  ;
    system.out.println("sum of input numbers is ");
```

```
int ans = obj.add(a, b);
System.out.println(ans);
}
}
```

Standard Library Methods.

- println() , sqrt() , floor() , pow() , min() , max()

Lecture:-14

What is an array?

```
int age= 30;
int age-1 = 12;
int age_2 = 14;
int age-3 = 10;
. .
```

→ Collection of similar types of data

→ Homogenous types of data.

→ data structure which stores a collection of times of Homogenous items.

→ Indexing → 0 based

→ Contiguous memory locations.

## Representation of Array

```
int [] ages;
String [] names;
float weight [];
```

## Syntax for array Declaration

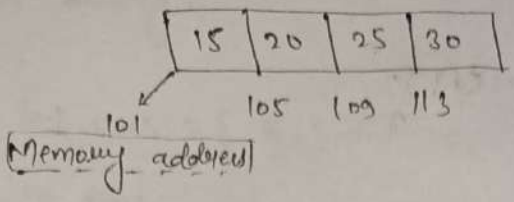
```
int [] ages = new int [10];
```

## Array literals:

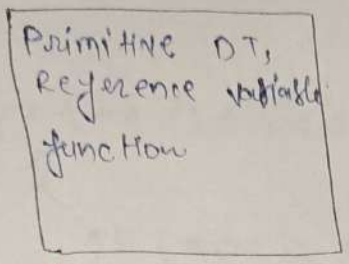
with curly braces we can initializes the array and add value do it during initialization without define the size.

```
int [] intArray = { 1, 2, 3, 4, 5, 6, 7 };
```

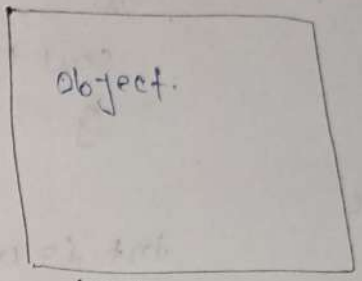
## Memory Allocation in Arrays.




total memory = 4 x 4 bytes




Stack memory




Heap memory

## Array types

(i) single dimensional Array

(ii) Multi-dimensional Array

```
int [] ages = new int [5]

int [] [] someages = new int [2] [3]
```

6 element

## Traversing through the Array

We can use loops to traverse through the Array, there are many ways to iterate over the arrays. The most common ways of looping through arrays in Java are

- for loop
- for each loop
- while loop

For Loop:

```
for( int i=0; i<3 ; i++)
{
  sout (ages[i]):
}
```

for Each :

```
for (int age : ages)
{
 sout ( ages):
}
```

while

```
int i = 0
while (i < 3) {
  sout (ages[i])
 i = i+1;
}
```

Question :- (1) Calculate the sum of all the elements in the given array.

Input: Array[] = { 1, 5, 3 }

Output :- 9.

```
int [] array = { 1, 5, 3 }
  int sum = 0
for( int i=0; i< array.lngth; i++)
{
   sum = sum + arr[i];
}
sout ( sum) ;
```

(2) Calculate the maximum value out of all the elements in the array.

```
int arr = { 5, 3, 6, 2, 8, 4 }
int Ans = 0;

for (int i = 0 ; i < arr.length ; i++)
  {
    If (arr[i] > Ans) {
        Ans = arr[i];
  } }

Sout. (Ans);
```

output:- 8

Q3 search the given element X in the array. If present then return the index else return -1.

Lecture :-15

## Taking Array input

```
Sout Println (" Enter size of Array");
int n = sc.nextInt();
int [] arr = new int[n];

sout.println ("Enter " + n + " elements");

for (int i=0 ; i < arr.length ; i++)
  {
    arr[i] = sc.next Int();
  }

for (int i=0; i<n ; i++) {
  system.out.println (arr[i] + " ");
  }
```

## Array reference in Java

In Java, an array reference means a variable that holds the memory address of an array, not the actual array values.

```
int []a= { 1,2,3 };
int [] b = a;  // both 'a' and 'b' refer to the same array.
b[0] = 99;
Sout. print ln ( a[0]);  // output is 99
```

## How to clone to Avoid reference sharing.

```
int [] a = { 1,2,3 };
int [] b = a.clone(); // Now b is a separate copy.
b[0] = 99;
Sout.println(a[0]); // output : 1 (not affected)
```

→ use .clone() or Arrays.copyOf() to create new Array.

* .copyOf() 2 parameter (i) kiska copy (arr)
(ii) length kitna (arr.length)

Ques:- count the number of occurrences of a particular element x.

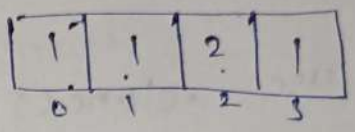
```
int x = sc.nextInt()
int count = 0
for(int i=0; i< Arr.length ; i++) {
    If (arr[i] == x) {
        count++;
    }
}
sout (count);
```

Ques:- Find the last occurrence of an element x in a given array.

```
int  lastindex = -1
int  x = sc.nextInt()

for(int i=0; i< Arr.length ;i++){

  if(arr[i] = x){
    lastindex = i;
  }
  };
sout (lastindex);
```




output : 3

Q Count the number of element strictly greater than value of x.

```
int  count = 0
int  x
```

① Check if the given array is sorted or not.

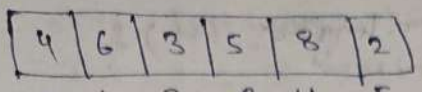
```
boolean check = true;
for(int i=1 ; i < arr.length; i++) {
  if( arr[i] < arr[i-1] ) {
     check = false;
     break;
  }
}
Sout (check)
```

→ static methods can only access static variable

→ A static method can only call other state method.

→ static method can't refer to non-static variable or methods.

Q Find the total number of pairs in the Array whose sum is equal to the given value X.




, Target = 7, ans = 2
(4, 3)
(5, 2)

Code

write full code taking input:

```
import java.util.Scanner;
Public class PairSum {
    Static int pairsum(int[] arr, int target) {
        int n = arr.length;
        int ans = 0;
    for(int i=0; i<n; i++) {  // first number
        for(int j=i+1; j<n; j++) {  // second number
            if(arr[i] + arr[j] == target) {
                Ans++;
            }
        }
    }
    return ans;
Public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.println("Enter array size");
    int n = sc.nextInt();
    int[] arr = new int[n];
```

```
System.out.println("Enter" + n + "elements");

for(int i=0; i<n; i++){
    arr[i] = sc.nextInt();
}

System.out.println("Enter target sum");
    int target = sc.nextInt();

System.out.println(pairsum(arr, target));
    }
}
```

Output:-

Enter Array size 6

Enter 6 elements

4 6 3 5 8 2

Target sum 7

2

Q. Count the number of triplets who sum is equal to the given value x.

```
Static int tripletsum (int[] arr, int target) {
        int ans = 0;
        int n = arr.length;
for (int i = 0; i < n; i++) {
    for (int j = i+1; j < n; j++) {
        for (int k = j+1; k < n; k++) {
            If (arr[i] + arr[j] + arr[k] == target) {
                ans = ans+1;
            }
        }
    }
}
return ans;
}
```

Output:-

Enter array size 5

Enter 5 elements

1 4 5 6 3

Enter target sum 12

→ 2

Pattern; Array manipulation (only positive elements in array)

Find the unique number in a given Array where all the elements are being repeated twice with one value being unique.

arr = | 1 | 2 | 3 | 4 | 2 | 1 | 3 |

Algorithm:- ① Traverse All pair
② equal pair → mark -1
③ Traverse array again,
value > 0 is our Answer.

```
Static int findunique(int[] arr) {
  int ans = -1
  int n = arr.length;
  for(int i=0; i<n ; i++) {
    for(int j = i+1 ; j<n ; j++) {
      if(arr[i] == arr[j]) {
        arr[i] = -1;
        arr[j] = -1;
      }
    }
  }
  for(int i=0; i<n ;i++){
    if(arr[i] > 0) {
      ans = arr[i]
    }
  }
  return ans;
}
```